

SIMATS ENGINEERING
ASSESSMENT TOOL 1

D. Madhu Sudhan
192524214

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

87.1

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
A Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
B Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
C Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
D Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
E Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
F Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
G Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
H Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

	A	B	C	D	E	F	G	H	
Q ₁	1.5	2	1.5	2	1	1	0.5	0.5	10
Q ₂	1	1.5	1.5	1.5	1	1	0.5	0.5	8.5
Q ₃	1.5	2	1.5	2	1	0.5	0.5	0.5	9.5
Q ₄	1	1	1	1	0.5	1	0.5	0.5	6.5
Q ₅	1.5	2	1	2	0.5	1	0.5	0.5	9

AB/5

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1.Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

Python Program to Generate Star Topology:

Aim:

To develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch and displays the host labels, switch name, and Cat6 UTP cable lengths.

Algorithm:

1. Read the number of hosts from the user.
2. Create a central switch named SW1.
3. Generate host names as H1, H2, H3, etc.
4. Assign a Cat6 UTP cable length to each host.
5. Display the connection between every host and the central switch.
6. Verify that all hosts are connected to the central switch.

Python Program:

```
def create_star_topology():
    try:
        n = int(input("Enter number of hosts: "))

        if n <= 0:
            print("Number of hosts must be greater than 0.")
            return

        switch = "SW1"

        print("\nSTAR TOPOLOGY")
        print("-" * 50)
        print(f"Central Switch: {switch}")
        print("-" * 50)

        connected_hosts = []
```

```

for i in range(1, n + 1):
    host = f'H{i}'

    # Example cable length for each host
    length = 10 + (i * 5)

    print(
        f'{host} --> {switch} | '
        f'Cat6 UTP Cable: {length} meters'
    )

    connected_hosts.append(host)

print('-' * 50)
print(f'Total Hosts Connected: {len(connected_hosts)}')

if len(connected_hosts) == n:
    print('Verification: All hosts are connected through the central
switch.')
else:
    print('Verification failed.')

except ValueError:
    print('Invalid input. Please enter an integer.')

```

create_star_topology()

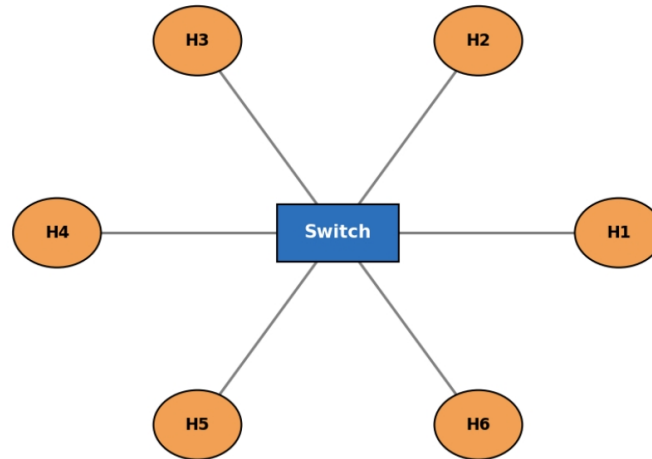
Sample Output:
Enter number of hosts: 4

STAR TOPOLOGY:
Central Switch: SW1
H1 --> SW1 | Cat6 UTP Cable: 15 meters
H2 --> SW1 | Cat6 UTP Cable: 20 meters
H3 --> SW1 | Cat6 UTP Cable: 25 meters
H4 --> SW1 | Cat6 UTP Cable: 30 meters
Total Hosts Connected: 4

Verification: All hosts are connected through the central switch.

Explanation:

Star Topology: Hosts Connected to a Central Switch



In a star topology, every host has a separate connection to a central switch. If one cable or host fails, the remaining hosts can continue communicating. However, failure of the central switch can affect the entire network.

2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

UTP Cable Segment Validation:

The question requires a function named `validate_segment(length_m)` that checks whether a UTP cable segment is within the 100-meter maximum specified in the question.

Aim:

To validate whether a given UTP cable segment satisfies the specified maximum length.

Python Program:

```
def validate_segment(length_m):
    if not isinstance(length_m, (int, float)):
        return False, "Invalid input. Cable length must be a number."

    if length_m < 0:
        return False, "Invalid cable length. Length cannot be negative."

    if length_m <= 100:
        return True, "Valid cable length."

    return False, "Invalid cable length. Maximum permitted length is 100 meters."

# Test cases
test_cases = [50, 100, 120]

for length in test_cases:
    result, message = validate_segment(length)

    print(f"Cable Length: {length} m")
    print(f"Result: {result}")
    print(f"Message: {message}")
    print()
```

Sample Output:

Cable Length: 50 m

Result: True

Message: Valid cable length.

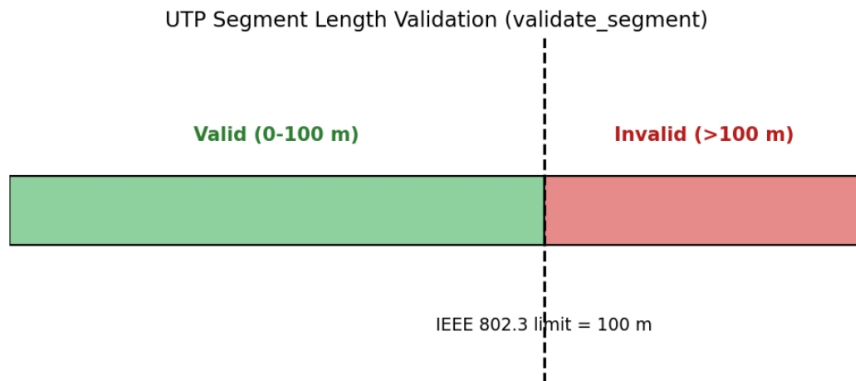
Cable Length: 100 m

Result: True

Message: Valid cable length.

Cable Length: 120 m

Result: False



Message: Invalid cable length. Maximum permitted length is 100 meters.

Interpretation:

- **50 m: Valid.**
- **100 m: Valid because it is exactly at the maximum.**
- **120 m: Invalid because it exceeds the specified limit.**

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks

CSMA/CD Simulation:

The third task requires simulation of four hosts connected through a central switch, including transmission attempts, collision detection, random back-off, and logging of events in `csma_log.txt`.

Aim:

To simulate the operation of the CSMA/CD protocol and record transmission attempts, collisions, and back-off intervals.

Python Program:

```
import random
import time
```

```
HOSTS = ["H1", "H2", "H3", "H4"]
LOG_FILE = "csma_log.txt"
```

```
def log_event(message):
    with open(LOG_FILE, "a") as file:
        file.write(message + "\n")
```

```
def transmit_frame(host, frame):
    attempt = 0
```

```
    while True:
        attempt += 1
```

```
        log_event(
            f'{host}: Attempt {attempt} to transmit {frame}'
        )
```

```
        print(
            f'{host}: Attempt {attempt} to transmit {frame}'
        )
```

```
        # Randomly decide whether collision occurs
        collision = random.choice([True, False])
```

```
        if collision:
            log_event(f'{host}: Collision detected')
            print(f'{host}: Collision detected')
```

```
        # Binary exponential backoff
        backoff_slots = random.randint(1, min(2 ** attempt, 16))
```

```
        log_event(
            f'{host}: Back-off interval = '
            f'{backoff_slots} slot(s)'
        )
```

```
        print(
            f'{host}: Back-off interval = '
            f'{backoff_slots} slot(s)'
        )
```

```

        time.sleep(0.1)
    else:
        log_event(
            f'{host}: Frame {frame} transmitted successfully'
        )

        print(
            f'{host}: Frame {frame} transmitted successfully'
        )

    break

# Clear previous log
open(LOG_FILE, "w").close()

print("CSMA/CD SIMULATION")
print("-" * 40)

for i, host in enumerate(HOSTS, start=1):
    frame = f'Frame-{i}'
    transmit_frame(host, frame)

print("-" * 40)
print(f"Simulation completed.")
print(f"Events are recorded in {LOG_FILE}")

```

Sample Output:

```

CSMA/CD SIMULATION
H1: Attempt 1 to transmit Frame-1
H1: Collision detected
H1: Back-off interval = 2 slot(s)
H1: Attempt 2 to transmit Frame-1
H1: Frame Frame-1 transmitted successfully

H2: Attempt 1 to transmit Frame-2
H2: Frame Frame-2 transmitted successfully

H3: Attempt 1 to transmit Frame-3
H3: Collision detected
H3: Back-off interval = 1 slot(s)
H3: Attempt 2 to transmit Frame-3
H3: Frame Frame-3 transmitted successfully

```

H4: Attempt 1 to transmit Frame-4
H4: Frame Frame-4 transmitted successfully

Simulation completed.

Events are recorded in csma_log.txt

csma_log.txt

H1: Attempt 1 to transmit Frame-1

H1: Collision detected

H1: Back-off interval = 2 slot(s)

H1: Attempt 2 to transmit Frame-1

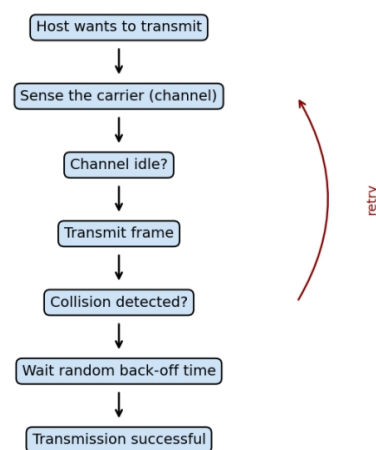
H1: Frame Frame-1 transmitted successfully

H2: Attempt 1 to transmit Frame-2

H2: Frame Frame-2 transmitted successfully

Explanation:

CSMA/CD Protocol Flow



CSMA/CD allows a host to sense the communication medium before transmission. When a collision is detected, the host stops transmitting, waits for a randomly selected back-off period, and then attempts transmission again.

Note: In modern switched full-duplex Ethernet, CSMA/CD is generally not required. This program is a simulation for the coding challenge.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC

Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

Client-Server Application with MAC Address Learning:

The fourth task requires a Python socket-based client-server application in which the server receives frames, forwards them, and maintains a simulated MAC address table.

Aim:

To simulate Ethernet switch communication using Python sockets and demonstrate MAC address learning and frame forwarding.

Server Program

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
mac_table = {}
```

```
def display_mac_table():
```

```
    print("\nMAC ADDRESS TABLE")
```

```
    print("-" * 45)
```

```
    if not mac_table:
```

```
        print("Table is empty.")
```

```
    else:
```

```
        for mac, port in mac_table.items():
```

```
            print(f"MAC Address: {mac:<20} Port: {port}")
```

```
    print("-" * 45)
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
server.bind((HOST, PORT))
```

```
server.listen(5)
```

```
print("Switch Server started.")
```

```

print(f"Listening on {HOST}:{PORT}")

while True:
    connection, address = server.accept()

    print(f"\nConnection received from {address}")

    data = connection.recv(1024).decode()

    if not data:
        connection.close()
        continue

    # Expected format:
    # source_mac,destination_mac,message
    parts = data.split(",", 2)

    if len(parts) != 3:
        connection.send("Invalid frame format.".encode())
        connection.close()
        continue

    source_mac, destination_mac, message = parts

    # Learn source MAC address
    mac_table[source_mac] = address[1]

    print(f"Received frame: {message}")
    print(f"Source MAC: {source_mac}")
    print(f"Destination MAC: {destination_mac}")

    # Forwarding decision
    if destination_mac in mac_table:
        response = (
            f"Frame forwarded to port "
            f"{mac_table[destination_mac]}"
        )
    else:
        response = "Destination unknown. Frame will be broadcast."

    connection.send(response.encode())

    display_mac_table()

```

```
connection.close()
```

Client Program:

```
import socket
```

```
SERVER_HOST = "127.0.0.1"
```

```
SERVER_PORT = 5000
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client.connect((SERVER_HOST, SERVER_PORT))
```

```
source_mac = input("Enter Source MAC Address: ")
```

```
destination_mac = input("Enter Destination MAC Address: ")
```

```
message = input("Enter Frame Data: ")
```

```
frame = f"{source_mac},{destination_mac},{message}"
```

```
client.send(frame.encode())
```

```
response = client.recv(1024).decode()
```

```
print("\nSwitch Response:")
```

```
print(response)
```

```
client.close()
```

Sample Output:

Switch Server started.

Listening on 127.0.0.1:5000

Connection received from ('127.0.0.1', 54021)

Received frame: Hello Host 2

Source MAC: AA:BB:CC:DD:EE:01

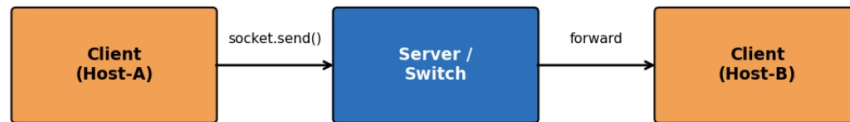
Destination MAC: AA:BB:CC:DD:EE:02

MAC ADDRESS TABLE

MAC Address: AA:BB:CC:DD:EE:01 Port: 54021

Explanation:

Client-Server Simulation with Learned MAC Address Table



MAC Address	Port
AA:BB:CC:00:00:01	Host-A
AA:BB:CC:00:00:02	Host-B

An Ethernet switch learns the source MAC address of an incoming frame and associates it with the port through which the frame arrived. When a destination MAC address is known, the switch can forward the frame through the corresponding port. If the destination MAC address is unknown, the frame can be flooded to other ports.

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

StarSwitch Class:

The fifth task requires a Python class called StarSwitch with `add_port()`, `remove_port()`, and `broadcast()` methods. It must maintain active ports and log broadcast operations.

Aim:

To implement a Python class that models the basic broadcasting functionality of an Ethernet switch in a star topology.

Python Program:

class StarSwitch:

```
def __init__(self):
    self.active_ports = []
    self.broadcast_log = []

def add_port(self, host_id):
    if host_id not in self.active_ports:
        self.active_ports.append(host_id)
        print(f'{host_id} connected to the switch.')
    else:
        print(f'{host_id} is already connected.')

def remove_port(self, host_id):
    if host_id in self.active_ports:
        self.active_ports.remove(host_id)
        print(f'{host_id} disconnected from the switch.')
    else:
        print(f'{host_id} is not connected.')

def broadcast(self, frame, source_host):
    if source_host not in self.active_ports:
        print(
            f'Broadcast failed: {source_host} "
            f'is not connected."
        )
        return

    destinations = [
        host for host in self.active_ports
        if host != source_host
    ]

    log_entry = (
        f'Source: {source_host}, "
        f'Frame: {frame}, "
        f'Destinations: {destinations}"
    )
```

```

self.broadcast_log.append(log_entry)

print("\nBroadcast Operation")
print("-" * 40)
print(f"Source Host: {source_host}")
print(f"Frame: {frame}")

if destinations:
    print("Frame received by:")

    for host in destinations:
        print(f" {host}")
    else:
        print("No other active hosts.")

def display_ports(self):
    print("\nACTIVE PORTS")
    print("-" * 30)

    if self.active_ports:
        for host in self.active_ports:
            print(host)
    else:
        print("No active ports.")

def display_log(self):
    print("\nBROADCAST LOG")
    print("-" * 50)

    for entry in self.broadcast_log:
        print(entry)

# Demonstration
switch = StarSwitch()

switch.add_port("H1")
switch.add_port("H2")
switch.add_port("H3")
switch.add_port("H4")

switch.display_ports()

```

```
switch.broadcast(  
    "Hello from H1",  
    "H1"  
)  
  
switch.broadcast(  
    "Network announcement",  
    "H3"  
)  
  
switch.remove_port("H4")  
  
switch.display_ports()  
  
switch.display_log()
```

Sample Output:

**H1 connected to the switch.
H2 connected to the switch.
H3 connected to the switch.
H4 connected to the switch.**

ACTIVE PORTS

**H1
H2
H3
H4**

Broadcast Operation:

**Source Host: H1
Frame: Hello from H1
Frame received by:
H2
H3
H4**

Broadcast Operation:

**Source Host: H3
Frame: Network announcement
Frame received by:
H1
H2**

H4

H4 disconnected from the switch.

ACTIVE PORTS

H1

H2

H3

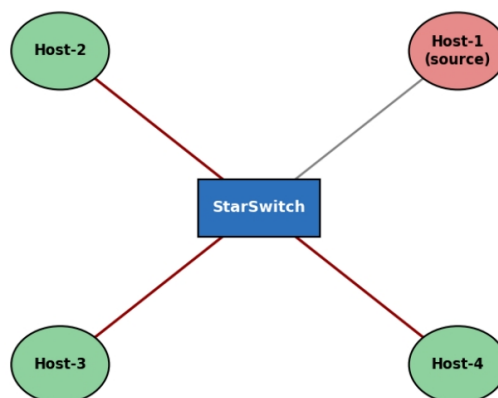
BROADCAST LOG:

Source: H1, Frame: Hello from H1, Destinations: ['H2', 'H3', 'H4']

Source: H3, Frame: Network announcement, Destinations: ['H1', 'H2', 'H4']

Explanation:

broadcast(frame): Source Sends to All Other Active Ports



Broadcasting is used when a frame needs to reach multiple hosts or when the destination MAC address is not known. The switch sends the broadcast frame to all active ports except the source port.

In a star topology, the central switch provides the connection point between hosts, making it possible to manage and forward network traffic efficiently.